

剧本 YAML Schema 定义文档

1. 文档概述

本文档用于说明《叙构》项目中的剧本 YAML Schema 设计。该 Schema 用于约束小说转剧本流程中生成的结构化剧本数据，确保模型生成结果、后端校验逻辑、前端展示逻辑以及后续导出流程之间拥有统一的数据格式。

该 Schema 的核心目标不是把剧本限制成唯一写法，而是为 AI 生成的剧本初稿提供一个稳定、可校验、可追踪、可转换的结构基础。通过该 Schema，系统可以将小说原文、改编参数、人物表、幕、场景、对白、动作说明和质量报告组织到同一份 YAML 文件中，方便后续进行校验、清洗、局部重渲染和 Word 文档导出。

2. Schema 基本信息

该 Schema 使用 JSON Schema Draft 2020-12 标准进行定义。

Schema 基本信息如下：

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "$id": "https://ai-novel-to-script.local/schemas/screenplay_schema.json",
  "title": "Screenplay YAML MVP Schema"
}
```

其中：

- `$schema` 表示当前 Schema 遵循 JSON Schema Draft 2020-12 规范。
- `$id` 为该 Schema 的唯一标识地址，用于在项目中标记和引用该结构定义。
- `title` 表示该 Schema 是剧本 YAML 的 MVP 版本结构定义。

该 Schema 虽然以 JSON Schema 形式描述，但实际服务对象是 YAML 格式的剧本数据。由于 YAML 可以被解析为对象结构，因此可以使用 JSON Schema 对其进行结构校验。

3. 顶层结构设计

Schema 的顶层类型为 object，并要求必须包含 `screenplay` 字段。

```
"required": ["screenplay"]
```

也就是说，任何符合该 Schema 的剧本 YAML，都必须以 `screenplay` 作为核心根节点。

设计原因

将所有剧本内容统一放在 `screenplay` 节点下，有几个好处：

第一，它让数据结构更加清晰。无论后续增加项目配置、版本信息、导出信息，还是调试信息，都不会和剧本主体混在一起。

第二，它方便前后端统一读取。前端、后端、校验器和导出器都可以从同一个入口访问剧本主体，降低解析复杂度。

第三，它为未来扩展留出了空间。顶层允许 `additionalProperties: true`，意味着未来可以在不破坏现有结构的情况下增加其他辅助字段。

4. screenplay 主体结构

`screenplay` 是整份剧本 YAML 的主体。它被定义为 object，并要求包含以下必要字段：

```
"required": ["meta", "source_novel", "characters", "acts", "quality_report"]
```

这五个字段分别对应：

- `meta`：剧本元信息
- `source_novel`：原小说来源信息
- `characters`：人物表
- `acts`：剧本幕与场景结构
- `quality_report`：质量报告

此外，`screenplay` 中还定义了可选字段：

- `title`：剧本标题
- `type`：剧本类型或媒介类型

设计原因

这五个必填字段覆盖了一份结构化剧本最基本的生命周期信息：

- `meta` 记录生成和配置背景。
- `source_novel` 保留原文来源。
- `characters` 保存人物识别与角色信息。
- `acts` 承载真正的剧本内容。
- `quality_report` 用于判断生成结果是否完整、是否覆盖原文、是否适合继续进入清洗和导出流程。

这种设计让剧本不只是最终文本，而是一份可以被继续处理的数据资产。

5. meta：剧本元信息

`meta` 被定义为 object，并允许额外字段：

```
"meta": {  
  "type": "object",
```

```
"additionalProperties": true
}
```

Schema 当前没有强制规定 `meta` 内部必须包含哪些字段。

设计原因

`meta` 的作用是保存剧本生成过程中的元数据，例如生成时间、模型信息、改编风格、适用场景、参数配置、版本号等。

之所以没有在 MVP 阶段强制限定 `meta` 字段，是因为项目仍处于可迭代状态，不同版本可能会记录不同类型的元信息。保留 `additionalProperties: true` 可以让后端在后续扩展时更加灵活，而不会因为新增字段导致校验失败。

6. source_novel: 原小说来源信息

`source_novel` 用于记录原小说相关信息。当前 Schema 中重点定义了一个字段：

```
"chapter_count": {
  "type": "integer",
  "minimum": 0
}
```

该字段表示原小说识别出的章节数量。

设计原因

小说转剧本并不是简单的文本改写，而是基于原文结构进行改编。章节数量是后续判断生成质量的重要基础。

例如：

- 系统可以根据章节数量判断原文是否足够完整。
- 生成剧本时可以尽量让场景覆盖不同章节。
- 质量报告可以根据章节覆盖情况判断剧本是否遗漏了重要内容。
- 前端可以在结构预览中展示章节识别结果。

`minimum: 0` 的设计允许空数据或异常输入进入校验流程，但不会允许负数这种明显不合理的值。

7. characters: 人物表

`characters` 被定义为数组，每个元素为一个人物对象。

人物对象中定义了以下字段：

```
"name": { "type": "string" },
"role": { "type": "string" },
"description": { "type": "string" }
```

其中：

- `name` 表示人物名称。
- `role` 表示人物在故事中的角色身份，例如主角、反派、配角等。
- `description` 表示人物简介、性格特征或剧情功能。

设计原因

人物是小说改编剧本时最重要的结构之一。剧本不是单纯地复述原文，而是通过人物的行动、对白和冲突推动故事。

设置 `characters` 字段有几个目的：

第一，方便前端在结构预览中展示人物表。

第二，方便后续校验剧本对白中的人物是否来自已识别人物。

第三，方便 AI 创作助手基于人物信息继续分析动机、关系和冲突。

第四，方便导出剧本时形成更完整的项目资料。

当前人物对象允许 `additionalProperties: true`，这意味着未来可以继续扩展人物字段，例如人物弧光、关系、年龄、身份标签、出场章节等。

8. acts：幕与场景结构

`acts` 是剧本的核心内容字段。它被定义为数组，并要求至少包含一个元素：

```
"acts": {
  "type": "array",
  "minItems": 1
}
```

每个 act 对象必须包含 `scenes` 字段。

```
"required": ["scenes"]
```

每一幕可以包含以下字段：

- `act_id`：幕 ID
- `title`：幕标题
- `summary`：幕摘要
- `scenes`：该幕下的场景列表

设计原因

使用 `acts` 作为剧本主体结构，是为了让剧本具备明确的叙事层级。

小说通常以章节组织，而剧本更常以幕、场景和节拍组织。通过 `acts`，系统可以把原文内容重新组织成更适合剧本阅读和制作的结构。

`acts` 至少包含一个元素，保证生成结果不是空剧本。每一幕必须包含 `scenes`，则保证剧本主体最终能够落到具体场景上，而不是只有抽象摘要。

9. scenes：场景结构

每一幕中的 `scenes` 字段是一个数组，并要求至少包含一个场景：

```
"scenes":{
  "type": "array",
  "minItems": 1
}
```

每个 scene 对象可以包含以下字段：

- `scene_id`：场景 ID
- `title`：场景标题
- `source_chapter_id`：来源章节 ID
- `interior_exterior`：内景或外景
- `slugline`：场景标题行
- `transition`：转场方式
- `location`：地点
- `time`：时间
- `summary`：场景摘要
- `characters`：场景出场人物
- `actions`：动作说明
- `beats`：场景节拍
- `dialogues`：对白列表

设计原因

场景是小说转剧本过程中最关键的中间单位。相比小说章节，场景更接近影视剧、舞台剧、广播剧和分镜脚本的组织方式。

这一层设计解决了几个问题：

第一，它让小说内容可以被拆解为可拍摄、可演出、可阅读的片段。

第二，它支持来源追踪。`source_chapter_id` 可以帮助用户知道某个场景来自原文哪一章，避免改编过程中丢失原文依据。

第三，它支持多媒介表达。`slugline`、`location`、`time`、`interior_exterior` 和 `transition` 更偏向影视剧格式；`actions` 和 `dialogues` 则可以服务于舞台剧、广播剧、有声书改编等不同形式。

第四，它支持局部重渲染。系统可以按幕或按场景定位内容，只重写某个场景，而不是重新生成整份剧本。

10. beats：场景节拍

`beats` 是场景内部的节拍列表。每个 beat 对象可以包含以下字段：

- `type`：节拍类型
- `content`：节拍内容
- `description`：描述
- `action`：动作内容
- `character`：关联人物
- `line`：台词内容

设计原因

`beats` 的设计是为了让剧本不只停留在“场景摘要”和“对白列表”两个层级之间。

在剧本创作中，节拍可以表示一个动作、一个情绪变化、一个信息揭示、一次冲突升级，或者一句关键台词。它比场景更细，但又比单纯对白更有结构。

保留 `beats` 字段有几个好处：

第一，它可以帮助系统理解场景内部的推进逻辑。

第二，它适合做局部润色和重写，例如只强化某个冲突节拍。

第三，它可以为未来分镜、节奏分析、冲突检测等功能提供基础。

第四，它让 AI 生成结果更容易被解释和调试，因为每个场景内部的叙事动作可以被拆开查看。

11. dialogues：对白结构

`dialogues` 是场景中的对白列表。每个对白对象可以包含：

- `character`：说话人物
- `line`：台词内容
- `parenthetical`：括号提示，例如语气、动作或情绪说明

设计原因

对白是剧本最重要的表达形式之一。将对白独立成数组，可以让系统更方便地进行格式化、清洗和导出。

这种设计有几个优势：

第一，方便在清洗剧本时将对白转换成标准剧本文本格式。

第二，方便后续进行人物对白统计。

第三，方便判断某个人物是否出场过、是否有台词。

第四，方便进行局部优化，例如只重写某个角色的台词风格。

`parenthetical` 字段的存在，让对白可以包含更细腻的表演提示，例如“低声”“迟疑”“笑着”等，使剧本文本更接近可表演状态。

12. actions：动作说明

`actions` 被定义为数组，但当前 Schema 没有限制其内部元素类型。

设计原因

动作说明在不同剧本类型中可能存在较大差异。

例如：

- 影视剧中，动作说明可能偏向镜头和行为。
- 舞台剧中，动作说明可能偏向调度和舞台提示。
- 广播剧中，动作说明可能偏向声音和氛围。
- 分镜初稿中，动作说明可能进一步拆分为镜头动作。

因此，MVP 阶段没有对 `actions` 做过度约束，而是保留一定自由度，让不同媒介类型的生成结果都能被容纳。

13. quality_report：质量报告

`quality_report` 是必填对象，并要求包含以下字段：

```
"required": [  
  "chapter_count",  
  "scene_count",  
  "character_count",  
  "chapter_coverage_rate"  
]
```

字段含义如下：

- `chapter_count`：章节数量
- `scene_count`：场景数量
- `character_count`：人物数量
- `chapter_coverage_rate`：章节覆盖率

其中，前三个字段均为非负整数：

```
"type": "integer",  
"minimum": 0
```

`chapter_coverage_rate` 可以是 0 到 1 之间的数字，也可以是字符串：

```
"oneOf": [
  { "type": "number", "minimum": 0, "maximum": 1 },
  { "type": "string" }
]
```

设计原因

`quality_report` 是该 Schema 中非常重要的一部分。它让剧本 YAML 不只是保存内容，还能保存系统对生成结果的基本判断。

通过质量报告，系统可以快速回答几个问题：

- 原文识别了多少章？
- 剧本生成了多少个场景？
- 识别出了多少个人物？
- 生成结果覆盖了多少原文章节？

这些信息可以用于前端展示，也可以用于判断是否允许进入下一步流程，例如 YAML 校验、剧本清洗或最终导出。

`chapter_coverage_rate` 允许数字或字符串，是为了兼容不同生成情况。例如在正常情况下，它可以是 `0.85` 这样的数值；在特殊情况下，也可以用字符串表示“未计算”“无法计算”或其他说明性状态。

14. additionalProperties 的设计

该 Schema 多处设置了：

```
"additionalProperties": true
```

这意味着对象中允许出现 Schema 没有明确列出的额外字段。

设计原因

这是一个非常重要的 MVP 设计选择。

对于 AI 生成内容来说，过于严格的 Schema 容易导致大量生成结果无法通过校验，尤其是在项目仍处于快速迭代阶段时。允许额外字段，可以让系统在保持核心结构稳定的同时，容纳模型生成的补充信息。

例如，未来可能会增加：

- 改编参数
- 模型名称
- 生成时间
- 人物关系
- 场景情绪
- 冲突强度
- 镜头建议
- 声音设计

- 舞台调度
- 导出状态

这些字段即使当前没有被 Schema 明确列出，也不会破坏整体校验流程。

因此，该 Schema 的设计思路不是“严格限制所有内容”，而是“强约束核心结构，弱约束扩展内容”。

15. Schema 的校验边界

该 Schema 主要校验结构是否满足系统运行的最低要求，包括：

- 是否包含 `screenplay` 根节点。
- `screenplay` 是否包含必要的五个核心字段。
- `acts` 是否至少包含一幕。
- 每一幕是否至少包含一个场景。
- `quality_report` 是否包含必要统计字段。
- 数量类字段是否为非负整数。
- 章节覆盖率是否为合理数字或字符串。

但该 Schema 不负责判断以下内容：

- 剧本文学质量是否足够好。
- 人物动机是否合理。
- 对白是否自然。
- 场景冲突是否充分。
- 改编是否忠实于原文。
- 是否符合某一具体行业的最终剧本格式标准。

这些内容需要通过 AI 生成策略、后端业务逻辑、人工编辑和后续质量评估共同完成。

16. 在项目流程中的作用

该 Schema 在《叙构》的小说转剧本流程中主要承担以下作用：

16.1 统一 AI 输出格式

AI 生成剧本时，需要按照该结构组织结果，避免输出完全不可解析的自由文本。

16.2 支撑 YAML 校验

生成 YAML 后，后端可以使用该 Schema 检查结构是否符合最低要求，并返回 pass、warning 或 error 等校验结果。

16.3 支撑结构预览

前端可以读取 `characters`、`acts`、`scenes` 和 `quality_report`，展示人物表、章节覆盖情况、场景数量和剧本结构。

16.4 支撑剧本清洗

系统可以根据 `acts`、`scenes`、`actions` 和 `dialogues`，把结构化 YAML 转换为更适合人类阅读和编辑的剧本文本。

16.5 支撑局部重渲染

由于剧本被拆成幕和场景，系统可以定位到某一幕或某一场，只替换局部内容，并保持其他内容不变。

16.6 支撑最终导出

当用户确认最终剧本后，系统可以基于清洗后的剧本文本导出 Word 文档。YAML 则作为结构化中间结果，保存了更完整的生成依据。

17. 设计总结

该剧本 YAML Schema 的设计核心可以概括为：

以 screenplay 为根节点，以 meta、source_novel、characters、acts 和 quality_report 为核心结构，以幕和场景承载剧本主体，以人物、动作、节拍和对白支撑可读剧本生成，以质量报告保证流程可校验。

它的重点不是追求一次性定义完整的行业级剧本标准，而是为 AI 小说转剧本产品提供一套足够稳定、足够清晰、又保留扩展空间的 MVP 数据结构。

这种设计适合《叙构》当前的产品目标：让小说改编过程不再是一次不可控的黑箱生成，而是变成一个可以查看结构、校验结果、局部修改、持续保存和最终交付的创作 workflow。